

```

extension SPHINCS.PublicKey {
    static func loadPEM(_ pemPath: String) throws -> SPHINCS.PublicKey {
        let s = String(decoding: try readFile(pemPath), as: UTF8.self)
        return try SPHINCS.PublicKey(pem: s)
    }
    var pem: String { (try? self.pemEncoded()) ?? (try! self.value(forKey:
"pem") as! String) }
    func savePEM(_ pemPath: String) throws { try writeText(pemPath, text:
self.pem) }
}

// Modes / randomness
enum Mode: String { case pure, prehash }
enum Rand: String { case det, rnd }

struct SignOptions {
    let mode: Mode
    let rand: Rand
    let context: Data?
    let prehashLen: Int // SHAKE-256 output length for prehash mode
}

private func prehashKind(_ len: Int) -> SPHINCS.PreHash { .shake256(length:
len) }

func slh_sign(sk: SPHINCS.SecretKey, message: Data, opts: SignOptions)
throws -> Data {
    switch opts.mode {
    case .pure:
        return (opts.rand == .rnd)
            ? try sk.signRandomized(message: message, context: opts.context)
            : try sk.signDeterministic(message: message, context:
opts.context)
    case .prehash:
        let ph = prehashKind(opts.prehashLen)
        return (opts.rand == .rnd)
            ? try sk.signPreHashRandomized(message: message, preHash: ph,
context: opts.context)
            : try sk.signPreHashDeterministic(message: message, preHash: ph,
context: opts.context)
    }
}

```

```

func slh_verify(pk: SPHINCS.PublicKey, message: Data, signature: Data, mode:
Mode, context: Data?, prehashLen: Int) throws -> Bool {
    switch mode {
    case .pure:
        return try pk.verify(message: message, signature: signature,
context: context)
    case .prehash:
        let ph = prehashKind(prehashLen)
        return try pk.verifyPreHash(message: message, signature: signature,
preHash: ph, context: context)
    }
}

// ----- utils for ctx/hex/armor/info -----

@inline(__always) func parseHex(_ s: String) -> Data? {
    var bytes = [UInt8](); bytes.reserveCapacity(s.count/2)
    var i = s.startIndex
    func hv(_ c: Character) -> UInt8? {
        switch c {
        case "0"... "9": return UInt8(c.asciiValue! -
Character("0").asciiValue!)
        case "a"... "f": return 10 + UInt8(c.asciiValue! -
Character("a").asciiValue!)
        case "A"... "F": return 10 + UInt8(c.asciiValue! -
Character("A").asciiValue!)
        default: return nil
        }
    }
    while i < s endIndex {
        let j = s.index(i, offsetBy: 2, limitedBy: s endIndex) ?? s endIndex
        if j <= i || (j == s endIndex && s.distance(from: i, to: j) < 2) {
break }
        let b = String(s[i..<j])
        guard let hi = hv(b.first!), let lo = hv(b.last!) else { return nil
}
        bytes.append((hi << 4) | lo); i = j
    }
    return Data(bytes)
}

@inline(__always) func loadCtx(_ arg: String?) throws -> Data? {
    guard let arg = arg else { return nil }

```

```

        if arg.hasPrefix("@") { return try readFile(String(arg.dropFirst())) }
        guard let d = parseHex(arg) else { throw CLIBError.badArgs("Invalid hex
in --ctx") }
        return d
    }

func hexdump(_ d: Data, width: Int = 16) -> String {
    var out = ""; let bytes = [UInt8](d)
    var i = 0
    while i < bytes.count {
        let line = bytes[i..

```

```

String(format: "%02x", $0) }.joined() }
func sha512hex(_ d: Data) -> String { SHA512.hash(data: d).compactMap {
String(format: "%02x", $0) }.joined() }

// ----- CLI plumbing -----

func usage() -> Never {
    print("""
slhdsa - SLH-DSA-SHAKE-256f CLI

USAGE:
    slhdsa gen --sk secret.pem --pk public.pem

    slhdsa sign --sk secret.pem --in msg.bin --out sig.bin [--mode
pure|prehash] [--rand det|rnd] [--ctx hex|@file] [--ph-len N]
    slhdsa verify --pk public.pem --in msg.bin --sig sig.bin [--mode
pure|prehash] [--ctx hex|@file] [--ph-len N]

    slhdsa sig info --sig signature.bin [--hexdump]
    slhdsa sig armor --sig signature.bin --out signature.asc

NOTES:
    • Parameter set is locked to SLH-DSA-SHAKE-256f.
    • Pre-hash is SHAKE256 only (default length 32 via --ph-len).
    • Context is optional (≤ 255 bytes). Use --ctx HEX or --ctx @path.
""")
    exit(0)
}

let argv = Array(CommandLine.arguments.dropFirst())
guard let cmd = argv.first else { usage() }

@inline(__always) func get(_ name: String) -> String? {
    if let i = argv.firstIndex(of: name), i+1 < argv.count { return
argv[i+1] }
    return nil
}
@inline(__always) func has(_ name: String) -> Bool { argv.contains(name) }

do {
    switch cmd {
    // ----- keygen
    case "gen":

```

```

guard let skPath = get("--sk"), let pkPath = get("--pk") else {
    throw CLIErrors.badArgs("gen requires --sk and --pk paths")
}
let kp = try SPHINCS.KeyPair(parameter: PARAM_SET)
try kp.secretKey.savePEM(skPath)
try kp.publicKey.savePEM(pkPath)
print("✅ Generated SLH-DSA-SHAKE-256f keypair")
print("    Secret: \(skPath)")
print("    Public: \(pkPath)")

// ---- sign
case "sign":
    guard let skPath = get("--sk"), let inPath = get("--in"), let
outPath = get("--out") else {
        throw CLIErrors.badArgs("sign requires --sk, --in, --out")
    }
    let sk = try SPHINCS.SecretKey.loadPEM(skPath)
    let msg = try readFile(inPath)
    let mode = Mode(rawValue: get("--mode") ?? "pure") ?? .pure
    let rand = Rand(rawValue: get("--rand") ?? "det") ?? .det
    let ctx = try loadCtx(get("--ctx"))
    if let c = ctx, c.count > 255 { throw CLIErrors.badArgs("context must
be ≤ 255 bytes") }
    let phLen = Int(get("--ph-len") ?? "32") ?? 32
    let opts = SignOptions(mode: mode, rand: rand, context: ctx,
prehashLen: phLen)
    let sig = try slh_sign(sk: sk, message: msg, opts: opts)
    try writeFile(outPath, data: sig)
    print("📝 Wrote signature to \(outPath)")

// ---- verify
case "verify":
    guard let pkPath = get("--pk"), let inPath = get("--in"), let
sigPath = get("--sig") else {
        throw CLIErrors.badArgs("verify requires --pk, --in, --sig")
    }
    let pk = try SPHINCS.PublicKey.loadPEM(pkPath)
    let msg = try readFile(inPath)
    let sig = try readFile(sigPath)
    let mode = Mode(rawValue: get("--mode") ?? "pure") ?? .pure
    let ctx = try loadCtx(get("--ctx"))
    if let c = ctx, c.count > 255 { throw CLIErrors.badArgs("context must
be ≤ 255 bytes") }

```

```

        let phLen = Int(get("--ph-len") ?? "32") ?? 32
        let ok = try slh_verify(pk: pk, message: msg, signature: sig, mode:
mode, context: ctx, prehashLen: phLen)
        if ok { print("✅ Signature verified") } else { print("❌ Signature
verification failed"); exit(1) }

// ---- sig subcommands
case "sig":
    let sub = (argv.count > 1) ? argv[1] : nil
    switch sub {
    case "info":
        guard let sigPath = get("--sig") else { throw
CLIErrors.badArgs("sig info requires --sig <file>") }
        let sig = try readFile(sigPath)
        print("Signature: \(sigPath)")
        print("Size: \(sig.count) bytes")
        print("SHA-256: \(sha256hex(sig))")
        print("SHA-512: \(sha512hex(sig))")
        let head = sig.prefix(32), tail = sig.suffix(32)
        print("Head(32): \(head.map { String(format: "%02x", $0)
}.joined())")
        print("Tail(32): \(tail.map { String(format: "%02x", $0)
}.joined())")
        if has("--hexdump") {
            print("\nHexdump:\n" + hexdump(sig))
        }

    case "armor":
        guard let sigPath = get("--sig"), let outputPath = get("--out")
else {
            throw CLIErrors.badArgs("sig armor requires --sig <file> --
out <file>")
        }
        let sig = try readFile(sigPath)
        let asc = armorSignature(sig)
        try writeText(outputPath, text: asc)
        print("✍️ Wrote ASCII armor → \(outputPath)")

    default:
        throw CLIErrors.badArgs("sig requires subcommand: info | armor")
    }

default:

```

```
        usage()
    }
} catch let e as CLLError {
    die(e.description)
} catch {
    die("\(error)")
}
```

Usage

This doesn't add any new package dependencies beyond `SwiftSPHINCS`; `CryptoKit` ships with macOS. If you want `sig dearmor` too (to go from `.asc` → `.bin`), I'll wire it up in 10 lines.

Nice Thanks.

Anytime. 🛠️

If you want to keep rolling, I can drop a tiny patch to add:

- `sig dearmor` (ASC → BIN)
- `sign --armor` (emit `.asc` directly)
- `verify --armor` (auto-dearmor on the fly)
- `sig fingerprint` (SHA-256/512 or BLAKE3 of the sig)

Say the word and I'll paste the exact diff.

Sounds good.

Boom. Here's a drop-in patch that adds:

- `sig dearmor` → `.asc` → `.bin`
- `sign --armor` → emits ASCII-armored `.asc`

- `verify --armor` → auto-dearmors (and also auto-detects armor if you forget the flag)
- `sig fingerprint` → `--alg sha256|sha512|blake3|all` (default: `all`). For BLAKE3 I wired in a tiny SwiftPM dep; import name is **Blake3**.

1) Package.swift – add Blake3 dep

2) Sources/slhdsa/main.swift – full file (copy/paste)

```
swiftimport Foundation
import SwiftSPHINCS
import CryptoKit
import Blake3

// =====
// slhdsa: SLH-DSA-SHAKE-256f CLI
// - gen: keygen → PEM
// - sign: pure | prehash(SHAKE256 len N), det|rnd, optional context, --armor
// - verify: pure | prehash, optional context, auto-dearmor (--armor optional)
// - sig info: size / SHA-256 / SHA-512 / head/tail [--hexdump]
// - sig armor: BIN → ASC
// - sig dearmor: ASC → BIN
// - sig fingerprint: --alg sha256|sha512|blake3|all (default: all)
// =====

enum CLIError: Error, CustomStringConvertible {
    case badArgs(String), io(String), crypto(String)
    var description: String {
        switch self {
            case .badArgs(let s): return "Argument error: \(s)"
            case .io(let s):      return "I/O error: \(s)"
            case .crypto(let s):  return "Crypto error: \(s)"
        }
    }
}

@inline(__always) func readFile(_ p: String) throws -> Data {
```

```

    try Data(contentsOf: URL(fileURLWithPath: p), options: .mappedIfSafe)
}
@inline(__always) func writeFile(_ p: String, data: Data) throws {
    try data.write(to: URL(fileURLWithPath: p), options: .atomic)
}
@inline(__always) func writeText(_ p: String, text: String) throws {
    try text.data(using: .utf8)!.write(to: URL(fileURLWithPath: p), options:
.atomic)
}
@inline(__always) func die(_ s: String) -> Never {
    FileHandle.standardError.write(("error: \(s)\n").data(using: .utf8)!)
    exit(2)
}

// Parameter set locked to SLH-DSA-SHAKE-256f
let PARAM_SET = SPHINCS.Parameter.shake_256f

// PEM helpers
extension SPHINCS.SecretKey {
    static func loadPEM(_ pemPath: String) throws -> SPHINCS.SecretKey {
        let s = String(decoding: try readFile(pemPath), as: UTF8.self)
        return try SPHINCS.SecretKey(pem: s)
    }
    // Library exposes PEM in 3.x; prefer encoder if available.
    var pem: String { (try? self.pemEncoded()) ?? (try! self.value(forKey:
"pem") as! String) }
    func savePEM(_ pemPath: String) throws { try writeText(pemPath, text:
self.pem) }
}
extension SPHINCS.PublicKey {
    static func loadPEM(_ pemPath: String) throws -> SPHINCS.PublicKey {
        let s = String(decoding: try readFile(pemPath), as: UTF8.self)
        return try SPHINCS.PublicKey(pem: s)
    }
    var pem: String { (try? self.pemEncoded()) ?? (try! self.value(forKey:
"pem") as! String) }
    func savePEM(_ pemPath: String) throws { try writeText(pemPath, text:
self.pem) }
}

// Modes / randomness
enum Mode: String { case pure, prehash }
enum Rand: String { case det, rnd }

```

```

struct SignOptions {
    let mode: Mode
    let rand: Rand
    let context: Data?
    let prehashLen: Int // SHAKE-256 output length for prehash mode
}

private func prehashKind(_ len: Int) -> SPHINCS.PreHash { .shake256(length:
len) }

func slh_sign(sk: SPHINCS.SecretKey, message: Data, opts: SignOptions)
throws -> Data {
    switch opts.mode {
    case .pure:
        return (opts.rand == .rnd)
            ? try sk.signRandomized(message: message, context: opts.context)
            : try sk.signDeterministic(message: message, context:
opts.context)
    case .prehash:
        let ph = prehashKind(opts.prehashLen)
        return (opts.rand == .rnd)
            ? try sk.signPreHashRandomized(message: message, preHash: ph,
context: opts.context)
            : try sk.signPreHashDeterministic(message: message, preHash: ph,
context: opts.context)
    }
}

func slh_verify(pk: SPHINCS.PublicKey, message: Data, signature: Data, mode:
Mode, context: Data?, prehashLen: Int) throws -> Bool {
    switch mode {
    case .pure:
        return try pk.verify(message: message, signature: signature,
context: context)
    case .prehash:
        let ph = prehashKind(prehashLen)
        return try pk.verifyPreHash(message: message, signature: signature,
preHash: ph, context: context)
    }
}

// ----- utils for ctx/hex/armor/info/fp -----

```

```

@inline(__always) func parseHex(_ s: String) -> Data? {
    var bytes = [UInt8](); bytes.reserveCapacity(s.count/2)
    var i = s.startIndex
    func hv(_ c: Character) -> UInt8? {
        switch c {
            case "0"... "9": return UInt8(c.asciiValue! -
Character("0").asciiValue!)
            case "a"... "f": return 10 + UInt8(c.asciiValue! -
Character("a").asciiValue!)
            case "A"... "F": return 10 + UInt8(c.asciiValue! -
Character("A").asciiValue!)
            default: return nil
        }
    }
    while i < s.endIndex {
        let j = s.index(i, offsetBy: 2, limitedBy: s.endIndex) ?? s.endIndex
        if j <= i || (j == s.endIndex && s.distance(from: i, to: j) < 2) {
break }
        let b = String(s[i..

```

```

        let ascii = line.map { (b) -> String in
            let u = UnicodeScalar(b)
            return (u.value >= 32 && u.value < 127) ? String(Character(u)) :
        },
        ".
    }.joined()
    out += "\(\offs) \(\hexs.padding(toLength: width*3-1, withPad: " ",
startingAt: 0)) |\(\ascii)|\n"
    i += width
}
return out
}

```

```

func b64wrap(_ s: String, wrap: Int = 64) -> String {
    var out = ""; out.reserveCapacity(s.count + s.count/wrap + 64)
    var col = 0
    for ch in s {
        if col == wrap { out.append("\n"); col = 0 }
        out.append(ch); col += 1
    }
    if !out.hasSuffix("\n") { out.append("\n") }
    return out
}

```

```

let SIG_HDR = "-----BEGIN SPHINCS SIGNATURE-----"
let SIG_FTR = "-----END SPHINCS SIGNATURE-----"

```

```

func armorSignature(_ sig: Data) -> String {
    SIG_HDR + "\n" + b64wrap(sig.base64EncodedString()) + SIG_FTR + "\n"
}

```

```

enum ArmorError: Error { case badArmor }

```

```

func dearmorSignature(_ text: String) throws -> Data {
    guard let hdrRange = text.range(of: SIG_HDR),
        let ftrRange = text.range(of: SIG_FTR),
        hdrRange.lowerBound < ftrRange.lowerBound else { throw
ArmorError.badArmor }
    let body = text[hdrRange.upperBound..

```

```

ArmorError.badArmor }
    return out
}

func maybeDearmor(_ data: Data, force: Bool) -> Data {
    // If --armor given OR header detected, try dearmor; otherwise return
    raw
    if force, let s = String(data: data, encoding: .utf8),
s.contains(SIG_HDR) {
        return (try? dearmorSignature(s)) ?? data
    }
    if let s = String(data: data, encoding: .utf8), s.contains(SIG_HDR) {
        return (try? dearmorSignature(s)) ?? data
    }
    return data
}

func sha256hex(_ d: Data) -> String { SHA256.hash(data: d).map {
String(format: "%02x", $0) }.joined() }
func sha512hex(_ d: Data) -> String { SHA512.hash(data: d).map {
String(format: "%02x", $0) }.joined() }
func blake3hex(_ d: Data) -> String { Blake3().hash(data: Array(d)).map {
String(format: "%02x", $0) }.joined() }

// ----- CLI plumbing -----

func usage() -> Never {
    print("""
slhdsa - SLH-DSA-SHAKE-256f CLI

USAGE:
    slhdsa gen --sk secret.pem --pk public.pem

    slhdsa sign --sk secret.pem --in msg.bin --out sig.bin [--mode
pure|prehash] [--rand det|rnd] [--ctx hex|@file] [--ph-len N] [--armor]
    slhdsa verify --pk public.pem --in msg.bin --sig sig.bin [--mode
pure|prehash] [--ctx hex|@file] [--ph-len N] [--armor]

    slhdsa sig info          --sig signature.bin [--hexdump]
    slhdsa sig armor         --sig signature.bin --out signature.asc
    slhdsa sig dearmor       --in signature.asc --out signature.bin
    slhdsa sig fingerprint --sig signature.bin [--alg
sha256|sha512|blake3|all]

```

NOTES:

- Parameter set is locked to SLH-DSA-SHAKE-256f.
- Pre-hash is SHAKE256 only (default length 32 via --ph-len).
- Context is optional (≤ 255 bytes). Use --ctx HEX or --ctx @path.

```
""")
    exit(0)
}

let argv = Array(CommandLine.arguments.dropFirst())
guard let cmd = argv.first else { usage() }
@inline(__always) func get(_ name: String) -> String? {
    if let i = argv.firstIndex(of: name), i+1 < argv.count { return
argv[i+1] }
    return nil
}
@inline(__always) func has(_ name: String) -> Bool { argv.contains(name) }

do {
    switch cmd {
    // ---- keygen
    case "gen":
        guard let skPath = get("--sk"), let pkPath = get("--pk") else {
            throw CLIErrors.badArgs("gen requires --sk and --pk paths")
        }
        let kp = try SPHINCS.KeyPair(parameter: PARAM_SET)
        try kp.secretKey.savePEM(skPath)
        try kp.publicKey.savePEM(pkPath)
        print("✅ Generated SLH-DSA-SHAKE-256f keypair")
        print("  Secret: \(skPath)")
        print("  Public: \(pkPath)")

    // ---- sign
    case "sign":
        guard let skPath = get("--sk"), let inPath = get("--in"), let
outPath = get("--out") else {
            throw CLIErrors.badArgs("sign requires --sk, --in, --out")
        }
        let sk = try SPHINCS.SecretKey.loadPEM(skPath)
        let msg = try readFile(inPath)
        let mode = Mode(rawValue: get("--mode") ?? "pure") ?? .pure
        let rand = Rand(rawValue: get("--rand") ?? "det") ?? .det
        let ctx = try loadCtx(get("--ctx"))
```

```

        if let c = ctx, c.count > 255 { throw CLIErrors.badArgs("context must
be ≤ 255 bytes") }
        let phLen = Int(get("--ph-len") ?? "32") ?? 32
        let opts = SignOptions(mode: mode, rand: rand, context: ctx,
prehashLen: phLen)
        let sig = try slh_sign(sk: sk, message: msg, opts: opts)
        if has("--armor") {
            try writeText(outPath, text: armorSignature(sig))
        } else {
            try writeFile(outPath, data: sig)
        }
        print("🖋️ Wrote signature to \(outPath)")

// ---- verify
case "verify":
    guard let pkPath = get("--pk"), let inPath = get("--in"), let
sigPath = get("--sig") else {
        throw CLIErrors.badArgs("verify requires --pk, --in, --sig")
    }
    let pk = try SPHINCS.PublicKey.loadPEM(pkPath)
    let msg = try readFile(inPath)
    let sigRaw = try readFile(sigPath)
    let sig = maybeDearmor(sigRaw, force: has("--armor"))
    let mode = Mode(rawValue: get("--mode") ?? "pure") ?? .pure
    let ctx = try loadCtx(get("--ctx"))
    if let c = ctx, c.count > 255 { throw CLIErrors.badArgs("context must
be ≤ 255 bytes") }
    let phLen = Int(get("--ph-len") ?? "32") ?? 32
    let ok = try slh_verify(pk: pk, message: msg, signature: sig, mode:
mode, context: ctx, prehashLen: phLen)
    if ok { print("✅ Signature verified") } else { print("❌ Signature
verification failed"); exit(1) }

// ---- sig subcommands
case "sig":
    let sub = (argv.count > 1) ? argv[1] : nil
    switch sub {
    case "info":
        guard let sigPath = get("--sig") else { throw
CLIErrors.badArgs("sig info requires --sig <file>") }
        let raw = try readFile(sigPath)
        let sig = maybeDearmor(raw, force: false)
        print("Signature: \(sigPath)")

```

```

        print("Size: \(sig.count) bytes")
        print("SHA-256: \(sha256hex(sig))")
        print("SHA-512: \(sha512hex(sig))")
        let head = sig.prefix(32), tail = sig.suffix(32)
        print("Head(32): \(head.map { String(format: "%02x", $0)
}.joined())")
        print("Tail(32): \(tail.map { String(format: "%02x", $0)
}.joined())")
        if has("--hexdump") {
            print("\nHexdump:\n" + hexdump(sig))
        }

    case "armor":
        guard let sigPath = get("--sig"), let outputPath = get("--out")
else {
            throw CLIErrors.badArgs("sig armor requires --sig <file> --
out <file>")
        }
        let sig = maybeDearmor(try readFile(sigPath), force: false)
        try writeText(outputPath, text: armorSignature(sig))
        print("🖋 Wrote ASCII armor → \(outputPath)")

    case "dearmor":
        guard let inPath = get("--in"), let outputPath = get("--out") else
{
            throw CLIErrors.badArgs("sig dearmor requires --in <asc> --
out <bin>")
        }
        let asc = String(decoding: try readFile(inPath), as: UTF8.self)
        let bin = try dearmorSignature(asc)
        try writeFile(outputPath, data: bin)
        print("🖋 Wrote binary signature → \(outputPath)")

    case "fingerprint":
        guard let sigPath = get("--sig") else { throw
CLIErrors.badArgs("sig fingerprint requires --sig <file>") }
        let raw = try readFile(sigPath)
        let sig = maybeDearmor(raw, force: false)
        let alg = (get("--alg") ?? "all").lowercased()
        switch alg {
        case "sha256":
            print(sha256hex(sig))
        case "sha512":

```

```

        print(sha512hex(sig))
    case "blake3":
        print(blake3hex(sig))
    case "all":
        print("sha256 \(sha256hex(sig))")
        print("sha512 \(sha512hex(sig))")
        print("blake3 \(blake3hex(sig))")
    default:
        throw CLIErrors.badArgs("unknown --alg \(alg); use
sha256|sha512|blake3|all")
    }

    default:
        throw CLIErrors.badArgs("sig requires subcommand: info | armor |
dearmor | fingerprint")
    }

    default:
        usage()
    }
} catch let e as CLIErrors {
    die(e.description)
} catch {
    die("\(error)")
}

```

Quick demo

If you want me to add `sig decode` to print a structured view if SwiftSPHINCS ever exposes a public signature format, I'll bolt that on too.

How do I use a diff file? It seems harder. Do I delete the - manually or add the + then save it?

Short answer: don't hand-edit the `-` and `+` lines. That's what tools are for. If you must do it manually, yes—delete the `-` lines, add the `+` lines, leave the rest—but that's misery. Use one of these three dead-simple methods:

Option 1: You're in a Git repo (best)

1. Save the diff as a file:
2. (If not a repo yet) init and commit your current state:
3. Dry-run check, then apply:

If it fails because files moved or context shifted, try:

1. Build, test, commit the result:

Revert it if you hate the result:

Email-style patches (made with `git format-patch`)? Use:

Option 2: Plain folder (no Git), use patch

1. Save the diff as `change.diff`.
2. From the project root, dry-run then apply:

If it can't find files, tweak the `-p` number (`-p0`, `-p2`, etc.).

Rule of thumb: the `p` number strips leading path components from the filenames inside the diff. Example:

- Diff shows `--- a/Sources/slhdsa/main.swift` / `+++ b/Sources/slhdsa/main.swift`
→ use `-p1` (drops the `a/` and `b/`)
- Diff shows `--- Sources/slhdsa/main.swift` → use `-p0`

Undo if needed:

Option 3: VS Code (drag-and-drop brain-off)

- Open the project in VS Code.
- Command Palette → "Apply Patch...", select the `.patch/.diff` file, confirm.
(If it complains about paths, you're in the wrong folder level—open the repo root.)

What those symbols mean (so you never wonder again)

- Lines starting with `---` = old file; `+++` = new file.
- `@@ ... @@` = a "hunk" (where in the file the changes land).

- Lines starting with `-` = remove these.
- Lines starting with `+` = add these.
- Lines with a space = unchanged context.

So if you *were* doing it by hand (again: don't), you'd remove `-` lines, add `+` lines, leave space-prefixed lines alone.

Tiny real example for your case

Let's pretend you've got a diff that renames the types and adds two subcommands. Apply it with Git:

If you're patch-shy, shove it through a dry run first. If it barks about `-p` levels with `patch`, change the number until the dry run succeeds.

If you want, send me the actual `.diff` you're holding and I'll tell you exactly which `-p` to use and whether `git apply` or `git am` is the right hammer.

I broke it. Working copy of <https://github.com/thecoolwinter/SwiftBlake3.git> resolved at 0.1.0
error: 'slhdsa-cli': Conflicting identity for swiftsphincs: dependency
'github.com/theapplegates/swiftsphincs' and dependency 'github.com/leif-ibsen/swiftsphincs' both point to the same package identity 'swiftsphincs'. error:
ExitCode(rawValue: 1) [0/1] Planning build thor3@Mac slhdsa-cli %

Yep, you tripped SwiftPM's identity rule. It lowercases the repo name to an "identity," and you've got **two** deps that both resolve to `swiftsphincs`:

- `github.com/theapplegates/SwiftSPHINCS`
- `github.com/leif-ibsen/SwiftSPHINCS`

Pick **one**. Not two. SwiftPM will hiss at you every time.

Here's the quickest fix, plus the "why didn't my `./slhdsa` run?" thing.

1) Fix Package.swift (keep exactly one SwiftSPHINCS)

Option A — use your fork (safe if you want to tweak code)

Option B — use upstream (clean if you don't need forked changes)

Important: make sure there is **only one** SwiftSPHINCS line total. If you still have a leftover line for the other URL, delete it.

Also: you don't need `Digest` anymore for this CLI; if it's still in `dependencies`, remove it.

2) Clear the resolver's confusion and rebuild

From the project root:

3) About./slhdsa "no such file or directory"

SwiftPM doesn't drop the binary in the project root. It lives in the build folder. Run it like:

If you want a top-level shortcut:

...or install it on your PATH:

4) Sanity check (one-liners)

If anything still squawks, paste your **current** `Package.swift` (just the dependencies/targets blocks) and I'll bullet-hole the exact line to delete.

I broke it. Working copy of <https://github.com/thecoolwinter/SwiftBlake3.git> resolved at 0.1.0
error: 'slhdsa-cli': Conflicting identity for swiftsphincs: dependency
'github.com/theapplegates/swiftsphincs' and dependency 'github.com/leif-
ibsen/swiftsphincs' both point to the same package identity 'swiftsphincs'. error:
ExitCode(rawValue: 1) [0/1] Planning build thor3@Mac slhdsa-cli %

You didn't "break it"; SwiftPM just spotted that you're trying to depend on **two** packages that both have the same identity: `swiftsphincs`. It lowercases the repo name and uses only the last path segment, so:

- `github.com/theapplegates/SwiftSPHINCS`
- `github.com/leif-ibsen/SwiftSPHINCS`

...both collide as `swiftsphincs` . Pick **one**. Not two.

Here's the quickest rescue:

Step 1 — Fix Package.swift (keep exactly one SwiftSPHINCS)

Option A — use your fork

Option B — use upstream (if you don't need fork-only tweaks)

🔍 Sanity check: make sure there is **only one** SwiftSPHINCS line in the `dependencies` array.

If you still see both URLs anywhere, delete one.

Also, if you still have `Digest` in your deps, you can remove it—this CLI doesn't need it anymore.

Step 2 — Nuke the stale resolver state

Step 3 — Run the binary from where it actually lives

SwiftPM puts it in `.build/release` :

If you want `./slhdsa` at project root:

Quick "am I fixed?" smoke test

If it still whines, paste your **current** `Package.swift` (just the `dependencies` + `targets` blocks). I'll point at the exact line that needs to go, with the righteous fury of a linter on espresso.

```
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:77:27: error: 'SecretKey' is not
a member type of struct 'SwiftSPHINCS.SPHINCS' 75 | private func prehashKind(_ len: Int) ->
SPHINCS.PreHash { .shake256(length: len) } 76 | 77 | func slh_sign(sk: SPHINCS.SecretKey,
message: Data, opts: SignOptions) throws -> Data { | - error: 'SecretKey' is not a
member type of struct 'SwiftSPHINCS.SPHINCS' 78 | switch opts.mode { 79 | case.pure:
/Users/thor3/Documents/slhdsa-
cli/.build/checkouts/SwiftSPHINCS/Sources/SwiftSPHINCS/SPHINCS.swift:21:15: note:
'SPHINCS' declared here 19 | typealias Words = [UInt32] 20 | 21 | public struct SPHINCS { | -
note: 'SPHINCS' declared here 22 | 23 | static func randomBytes(_ n: Int) -> Bytes {
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:91:29: error: 'PublicKey' is not
a member type of struct 'SwiftSPHINCS.SPHINCS' 89 | } 90 | 91 | func slh_verify(pk:
SPHINCS.PublicKey, message: Data, signature: Data, mode: Mode, context: Data?,
```

```

prehashLen: Int) throws -> Bool { | - error: 'PublicKey' is not a member type of
struct 'SwiftSPHINCS.SPHINCS' 92 | switch mode { 93 | case.pure:
/Users/thor3/Documents/slhdsa-
cli/.build/checkouts/SwiftSPHINCS/Sources/SwiftSPHINCS/SPHINCS.swift:21:15: note:
'SPHINCS' declared here 19 | typealias Words = [UInt32] 20 | 21 | public struct SPHINCS { | -
note: 'SPHINCS' declared here 22 | 23 | static func randomBytes(_ n: Int) -> Bytes {
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:236:30: error: type 'SPHINCS'
has no member 'KeyPair' 234 | throw CLSError.badArgs("gen requires --sk and --pk paths")
235 | } 236 | let kp = try SPHINCS.KeyPair(parameter: PARAM_SET) | - error: type
'SPHINCS' has no member 'KeyPair' 237 | try kp.secretKey.savePEM(skPath) 238 | try
kp.publicKey.savePEM(pkPath) /Users/thor3/Documents/slhdsa-
cli/Sources/slhdsa/main.swift:248:30: error: type 'SPHINCS' has no member 'SecretKey' 246 |
throw CLSError.badArgs("sign requires --sk, --in, --out") 247 | } 248 | let sk = try
SPHINCS.SecretKey.loadPEM(skPath) | - error: type 'SPHINCS' has no member
'SecretKey' 249 | let msg = try readFile(inPath) 250 | let mode = Mode(rawValue: get("--
mode"))?? "pure")??pure /Users/thor3/Documents/slhdsa-
cli/Sources/slhdsa/main.swift:269:30: error: type 'SPHINCS' has no member 'PublicKey' 267 |
throw CLSError.badArgs("verify requires --pk, --in, --sig") 268 | } 269 | let pk = try
SPHINCS.PublicKey.loadPEM(pkPath) | - error: type 'SPHINCS' has no member
'PublicKey' 270 | let msg = try readFile(inPath) 271 | let sigRaw = try readFile(sigPath)
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:194:39: error: static member
'hash' cannot be used on instance of type 'Blake3' 192 | func sha256hex(_ d: Data) -> String
{ SHA256.hash(data: d).map { String(format: "%02x", $0) }.joined() } 193 | func sha512hex(_
d: Data) -> String { SHA512.hash(data: d).map { String(format: "%02x", $0) }.joined() } 194 |
func blake3hex(_ d: Data) -> String { Blake3().hash(data: Array(d)).map { String(format:
"%02x", $0) }.joined() } | - error: static member 'hash' cannot be used on instance
of type 'Blake3' 195 | 196 | // ----- CLI plumbing ----- thor3@Mac slhdsa-cli %

```

Let me work what you wrote while I wrote.

This Step 3 did not work thor3@Mac slhdsa-cli %./.build/release/slhdsa --help zsh: no such file or directory:./.build/release/slhdsa thor3@Mac slhdsa-cli %./.build/release/slhdsa --help zsh: no such file or directory:./.build/release/slhdsa thor3@Mac slhdsa-cli % ln -sf.build/release/slhdsa./slhdsa./slhdsa sig info --sig sig.bin zsh: no such file or directory:./slhdsa thor3@Mac slhdsa-cli %

Right now there is no `./.build/release/slhdsa` because your build didn't finish. No binary ⇒ nothing to run/symlink. Let's fix the compile errors and then Step 3 will Just Work™.

Why it failed

Your errors show calls like `SPHINCS.Parameter`, `SPHINCS.SecretKey`, etc. In the current SwiftSPHINCS API those types are **top-level** — not nested under `SPHINCS`. That's why the compiler is yelling " `SPHINCS` has no member ...".

Also, your BLAKE3 helper is calling an instance method that doesn't exist; the SwiftBlake3 API exposes a **static** `hash`.

Quick patch (one-liners)

Run these from the repo root:

(If you prefer to eyeball it instead of sed'ing, make these edits manually:)

- `let PARAM_SET = Parameter.shake_256f`
- `extension SecretKey { ... }` and `extension PublicKey { ... }` (no `SPHINCS.`)
- `func slh_sign(sk: SecretKey, ...)` and `func slh_verify(pk: PublicKey, ...)`
- `private func prehashKind(_ len: Int) -> PreHash { .shake256(length: len) }`
- `func blake3hex(_ d: Data) -> String { Blake3.hash(data: Array(d)) ... }`

Build & run (the part Step 3 was trying to do)

Now do:

If you want to iterate without hunting for the binary:

And if you still like a local launcher:

Sanity checks if something still moans

- See what actually got built:

- Make sure the product name is `slhdsa` (from your `Package.swift` target). If it isn't, use:
- If the compiler *still* complains about types, peek at what the package exports:

Once those edits land, the binary will exist and Step 3 won't faceplant. If you hit any new red ink, paste the first 20 lines of the error and I'll smack it down.

Why do I have 2 main.swift files? They look the same

You probably don't "have two mains"; you have two files named `main.swift` in different places, and only one of them is actually compiled. SwiftPM only cares about files under the target path you set in `Package.swift`:

If you've got another `main.swift` somewhere else (root folder, an old `Sources/slhdsa-cli/`, or inside `.build`), it'll just sit there looking important while doing nothing.

Here's how to find out what's real vs decoy:

1. list every `main.swift` (ignore build junk)
2. if you see more than one, diff them so you know which is which
3. the **only** one that matters is:

Everything else named `main.swift` is either a stray copy or from another target. Delete or rename the extra to something harmless (e.g., `sig_commands.swift`) so you don't confuse your future self.

Pro tip to confirm which file is being compiled:

It'll print the exact paths Swift is compiling. If it shows only `Sources/slhdsa/main.swift`, that's your real entry point.

Also: an executable target can have **one** entry point total: either a single `main.swift` **or** a type marked `@main`. If you end up with two `main.swift` in the same target, the compiler will eventually throw a "multiple entry points" tantrum. Keep one. Rename the rest.

```

swift build -c release gave me some errors. Almost forgot to copy the error
/Users/thor3/Documents/slhdsa-
cli/.build/checkouts/SwiftSPHINCS/Sources/SwiftSPHINCS/SPHINCS.swift:21:15: note:
'SPHINCS' declared here 19 | typealias Words = [UInt32] 20 | 21 | public struct SPHINCS { | -
note: 'SPHINCS' declared here 22 | 23 | static func randomBytes(_ n: Int) -> Bytes {
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:91:29: error: 'PublicKey' is not
a member type of struct 'SwiftSPHINCS.SPHINCS' 89 | } 90 | 91 | func slh_verify(pk:
SPHINCS.PublicKey, message: Data, signature: Data, mode: Mode, context: Data?,
prehashLen: Int) throws -> Bool { | - error: 'PublicKey' is not a member type of
struct 'SwiftSPHINCS.SPHINCS' 92 | switch mode { 93 | case.pure:
/Users/thor3/Documents/slhdsa-
cli/.build/checkouts/SwiftSPHINCS/Sources/SwiftSPHINCS/SPHINCS.swift:21:15: note:
'SPHINCS' declared here 19 | typealias Words = [UInt32] 20 | 21 | public struct SPHINCS { | -
note: 'SPHINCS' declared here 22 | 23 | static func randomBytes(_ n: Int) -> Bytes {
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:236:30: error: type 'SPHINCS'
has no member 'KeyPair' 234 | throw CLSError.badArgs("gen requires --sk and --pk paths")
235 | } 236 | let kp = try SPHINCS.KeyPair(parameter: PARAM_SET) | - error: type
'SPHINCS' has no member 'KeyPair' 237 | try kp.secretKey.savePEM(skPath) 238 | try
kp.publicKey.savePEM(pkPath) /Users/thor3/Documents/slhdsa-
cli/Sources/slhdsa/main.swift:248:30: error: type 'SPHINCS' has no member 'SecretKey' 246 |
throw CLSError.badArgs("sign requires --sk, --in, --out") 247 | } 248 | let sk = try
SPHINCS.SecretKey.loadPEM(skPath) | - error: type 'SPHINCS' has no member
'SecretKey' 249 | let msg = try readFile(inPath) 250 | let mode = Mode(rawValue: get("--
mode"))?? "pure")??pure /Users/thor3/Documents/slhdsa-
cli/Sources/slhdsa/main.swift:269:30: error: type 'SPHINCS' has no member 'PublicKey' 267 |
throw CLSError.badArgs("verify requires --pk, --in, --sig") 268 | } 269 | let pk = try
SPHINCS.PublicKey.loadPEM(pkPath) | - error: type 'SPHINCS' has no member
'PublicKey' 270 | let msg = try readFile(inPath) 271 | let sigRaw = try readFile(sigPath)
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:75:52: error: type 'PreHash'
has no member 'shake256' 73 | } 74 | 75 | private func prehashKind(_ len: Int) -> PreHash
{.shake256(length: len) } | - error: type 'PreHash' has no member 'shake256' 76 | 77 |
func slh_sign(sk: SPHINCS.SecretKey, message: Data, opts: SignOptions) throws -> Data {
thor3@Mac slhdsa-cli % swift build -c release

```

You're getting those errors because you're using the **theapplegates/SwiftSPHINCS** fork (per your `swift package resolve` output), but your code still assumes types are *nested*

under `SPHINCS.`. In the fork (and in upstream 3.4.x), the public types are **top-level** — i.e. `KeyPair`, `SecretKey`, `PublicKey`, `Parameter`, `PreHash` — **not** `SPHINCS.SecretKey`, etc. Also, your prehash helper is calling the wrong SHAKE API.

Let's fix all of that in one go.

1) Make sure you only depend on one SPHINCS package

You already reset the workspace, good. Keep **theapplegates/SwiftSPHINCS** (or switch to upstream `leif-ibsen/SwiftSPHINCS` — either works, but don't mix both).

If you want to lock to upstream instead, set this in `Package.swift` and re-resolve:

Docs for that package (feature list, parameter sets, PEM, etc.) are here, matching what you want.

2) Kill the SPHINCS. prefixes in your CLI

Your current build log shows lines like:

Those all need to become **top-level types**.

Run these edits (safe to paste as-is from repo root):

(You tried a broader `sed` before, but your error log shows some `SPHINCS.` survived. The above is surgical.)

3) Use the correct prehash helper (CryptoKit/Swift Crypto SHAKE256)

Your previous helper used `Digest`'s `SHAKE` with a wrong signature. Use Swift Crypto's XOF instead:

At the top of `Sources/slhdsa/main.swift` ensure:

(If you used `import CryptoKit` earlier and it worked, fine — but with SPM + swift-crypto, `import Crypto` is the right module.)

Replace your prehash with:

That removes all `SHAKE(.shake256).digest(...)` nonsense and matches the `Crypto 3.x` API.

4) Set the parameter set correctly

You changed this to `Parameter.shake_256f`, which is correct **if** the package exposes `enum Parameter { case shake_256f, ... }`. Your error "`cannot find 'Parameter' in scope`" happened because the `SPHINCS.` prefix was removed earlier but **other** references still had it or your import order was off.

After step 2, this line should be exactly:

(If your local package used a different name, run `grep -R "enum Parameter" .build/checkouts/SwiftSPHINCS/Sources -n` to confirm. In current releases, `shake_256f` exists.)

5) Fix the BLAKE3 helper (you already started this)

The error said: *"static member 'hash' cannot be used on instance of type 'Blake3'"*. Correct call looks like:

You already ran a `sed` for this — good.

6) Delete the stray root-level main.swift

You asked "why do I have 2 main.swift files?" You've got `main.swift` at repo root **and** `Sources/slhdsa/main.swift`. SwiftPM only builds the one under `Sources/slhdsa/`, but the duplicate is confusing and can trick your automated `sed` edits.

Kill it:

7) Clean, resolve, build

If the build succeeds, your binary will finally exist at:

If you see “no such file or directory” again, it means the build failed — scroll a little higher in that same terminal for the compiler error that stopped it.

8) Sanity check your calls with the top-level API

With the fixes above, these should compile:

```
swift// Generate
let kp = try KeyPair(parameter: PARAM_SET)           // PARAM_SET = .shake_256f
try kp.secretKey.savePEM("sk.pem")
try kp.publicKey.savePEM("pk.pem")

// Load
let sk = try SecretKey.loadPEM("sk.pem")
let pk = try PublicKey.loadPEM("pk.pem")

// Sign (pure)
let sig = try sk.sign(message: msgData)              // deterministic by default

// Sign (randomized)
let sigR = try sk.signRandomized(message: msgData)

// Sign (pre-hash, SHAKE256-L bytes), with optional context (≤255 bytes)
let digest = prehashSHAKE256(msgData, outLen: 32)
let sigPH = try sk.signPreHash(digest: digest, prehash: .shake256(length:
32), context: ctxData)

// Verify (pure)
let ok = pk.verify(message: msgData, signature: sig)

// Verify (pre-hash)
let okPH = pk.verifyPreHash(digest: digest, prehash: .shake256(length: 32),
signature: sigPH, context: ctxData)
```

If your package uses a slightly different prehash case name, swap `.shake256(length:)` for whatever it exposes (you can `grep -R "enum PreHash"` `.build/checkouts/SwiftSPHINCS/Sources -n` to see exact case names fast).

Why those original errors happened (in plain English)

- **“‘SecretKey’ is not a member type of struct ‘SPHINCS’”** — because `SecretKey` isn’t nested; it’s a top-level type.
 - **“‘type ‘SPHINCS’ has no member ‘KeyPair’ / ‘Parameter’ / ‘PublicKey’”** — same reason.
 - **“‘type ‘PreHash’ has no member ‘shake256’”** — your helper referenced the wrong SHAKE API; fix the helper and/or use the package’s `PreHash` cases exactly as defined.
 - **No binary at `./build/release/slhdsa`** — build never completed, so nothing got emitted.
-

If anything still barks after this, paste the **first** 15–20 lines of the new compiler error and I’ll zero in on it. (And yes, if you want me to drop a ready-to-paste patch that adds `sig info` / `sig armor` subcommands into your CLI, say the word — I’ll ship it.)

Step 7 error error: exhausted attempts to resolve the dependencies graph, with the following dependencies unresolved: * 'swiftsphincs' from <https://github.com/leif-ibsen/SwiftSPHINCS.git> I have a fork of an identical site. Can we use, <https://github.com/colinxu2020/slhdsa.git>

Short version: your resolver is still hunting for **leif-ibsen/SwiftSPHINCS**, which means your project still references it somewhere. And no, `colinxu2020/slhdsa` won’t help here—that repo is a **Python** SLH-DSA implementation, not a Swift package, so it can’t fix your SwiftPM mess. (Great project, wrong ecosystem.)

Here’s the fastest way out, pick A or B:

A) Keep the Swift CLI (recommended). Fix Package.swift and purge the stale pin

1. Make sure you only depend on **one** SPHINCS package. Use either your fork or upstream, not both.

Option A1 — your fork (you were already on this)

Option A2 — upstream only

(Upstream's README confirms it supports keygen/sign/verify/PEM & all parameter sets, which is exactly what you want.)

1. Make sure there's **not** a second SwiftSPHINCS line hiding somewhere:

If you see both URLs, delete one.

1. Nuke stale state and resolve again:
2. Still seeing SwiftPM try to fetch `leif-ibsen/SwiftSPHINCS` even though you removed it? Put a mirror in front of it and force the URL rewrite:

(Yes, "set-mirror" is a real thing; it rewrites dependency URLs for the resolver.)

1. Build:

If there's **no** binary, the build failed; scroll up for the compiler error that stopped it.

B) If you actually want to switch to Python

Totally different project path. `colinuxu2020/slhdsa` is a Python lib/CLI. You'd:

Then use its Python API/CLI to keygen/sign/verify. That's not your Swift CLI, so none of your Swift code applies. (The PyPI page matches that repo.)

Bonus: why your Step 7 error happened

SwiftPM complained:

That only happens if **somewhere** in your graph it still sees that URL. Either:

- It's still in `Package.swift`, or
- A mirror points to it, or
- You have a stale `Package.resolved` pin, or
- You're opening a different folder than the one you edited (it happens).

The grep + reset + (if needed) mirror commands above remove all four possibilities.

If you paste your **current** `Package.swift` (just the `dependencies` and `targets` blocks), I'll mark the exact line that needs to go. And if you *do* want the Python route instead, say the word and I'll whip up an equivalent `slhdsa.py` CLI with the same subcommands you've been using in Swift.

Let me go back and run some commands again. I had unsaved data in `~/Documents/slhdsa-cli/Sources/slhdsa/main.swift`

Good catch on the unsaved file. Before you re-run anything, do this 60-second sanity checklist so we don't chase ghosts.

0) Save the right file

You want the one at:

If you had any other `main.swift` lying around, ignore/delete it.

1) (Optional but recommended) Nuke the unused SHAKE helper

In your current file I see this:

You're not calling it anymore (we use the library's `PreHash` enum), and on some toolchains `SHAKE256` isn't exposed by `Crypto`. Delete that whole function to avoid a pointless build failure.

2) Verify the SPHINCS types are top-level

Your file already shows this (good):

If you still see `SPHINCS.SecretKey` / `SPHINCS.PublicKey` anywhere, change them to `SecretKey` / `PublicKey`.

3) Double-check the exact PreHash case name

Some versions use slightly different case spelling. Quick lookup:

Then set this line to match whatever you find:

If the enum shows `case shake_256(length: Int)` then use:

4) Make sure you only depend on one SwiftSPHINCS

Open `Package.swift` and keep **either** your fork **or** upstream, not both.

Your fork example:

Upstream example:

Also keep the others you need:

and in the target deps:

5) Clean & resolve (stale pins are the devil)

If it STILL insists on the other URL, add a resolver mirror (only if needed):

6) Build & run

If `./.build/release/slhdsa` doesn't exist, the build failed. Scroll up and grab the **first** error — that's the one that matters.

7) Quick smoke test

If anything barks after this, paste the first 15–20 lines of the error and we'll smack it down.

You see this, `@inline(__always) func prehashSHAKE256(_ msg: Data, outLen: Int) -> Data {`
`var xof = SHAKE256() xof.update(data: msg) let bytes = xof.finalize(outputByteCount: outLen)`
`return Data(bytes) }` In `main.swift`? I don't see it.

Good eye — and no, you're not missing it. That `prehashSHAKE256(...)` helper is **not** in your current `main.swift` (and that's fine). In this version we switched to the library's own pre-hash selector via:

So there's no need for a separate SHAKE XOF helper anymore. If you're seeing references to `prehashSHAKE256` in old snippets or terminal history, that's just the previous approach. Ignore it.

What *does* need fixing in the file you showed are the lingering `SPHINCS.` prefixes and one Blake3 call. Do these tiny edits and you should finally build:

Replace these prefixes (top-level types, not nested)

- `let PARAM_SET = **SPHINCS.**Parameter.shake_256f` → `let PARAM_SET = Parameter.shake_256f`
- `extension **SPHINCS.**SecretKey` → `extension SecretKey`
- `extension **SPHINCS.**PublicKey` → `extension PublicKey`
- `private func prehashKind(_ len: Int) -> **SPHINCS.**PreHash { ... }` → `private func prehashKind(_ len: Int) -> PreHash { ... }`
- `func slh_sign(sk: **SPHINCS.**SecretKey, ...)` → `func slh_sign(sk: SecretKey, ...)`
- `func slh_verify(pk: **SPHINCS.**PublicKey, ...)` → `func slh_verify(pk: PublicKey, ...)`
- `let kp = try **SPHINCS.**KeyPair(parameter: PARAM_SET)` → `let kp = try KeyPair(parameter: PARAM_SET)`
- `let sk = try **SPHINCS.**SecretKey.loadPEM(skPath)` → `let sk = try SecretKey.loadPEM(skPath)`
- `let pk = try **SPHINCS.**PublicKey.loadPEM(pkPath)` → `let pk = try PublicKey.loadPEM(pkPath)`

Fix Blake3 call

- `func blake3hex(_ d: Data) -> String { **Blake3().hash(data: Array(d)) ... }`
→ `func blake3hex(_ d: Data) -> String { Blake3.hash(data: Array(d)) ... }`

If you like doing it the lazy way, from your project root:

Now rebuild:

Optional: bring the old helper back (only if you really want it)

Not needed, but if you prefer an explicit XOF:

Then you'd sign/verify with the digest form of the APIs. But again: **your current code already uses** `PreHash.shake256(length:)`, so you don't need this.

If anything else screams red, toss me the first 15 lines of the error and I'll whack it.

What does this mean? It has 2 arrows. Do I use those? `private func prehashKind(_ len: Int) -> **SPHINCS.**PreHash { ... }` → `private func prehashKind(_ len: Int) -> PreHash { ... }`

The arrows mean “replace this with that.” They’re not code you literally paste. And the **bold** was just me yelling politely.

So your line:

should become exactly:

That’s it—drop the `SPHINCS.` prefix, keep the rest. Also, your editor shows the call cut off at `le...`; make sure it’s the full `length:` label.

While you’re here, you’ve got a few more `SPHINCS.` leftovers in this file. Nuke the prefix on these too:

- `func slh_sign(sk: SPHINCS.SecretKey, ...) → func slh_sign(sk: SecretKey, ...)`
- `func slh_verify(pk: SPHINCS.PublicKey, ...) → func slh_verify(pk: PublicKey, ...)`
- `let kp = try SPHINCS.KeyPair(parameter: PARAM_SET) → let kp = try KeyPair(parameter: PARAM_SET)`
- `let sk = try SPHINCS.SecretKey.loadPEM(skPath) → let sk = try SecretKey.loadPEM(skPath)`
- `let pk = try SPHINCS.PublicKey.loadPEM(pkPath) → let pk = try PublicKey.loadPEM(pkPath)`
- `extension SPHINCS.PublicKey { ... } → extension PublicKey { ... }`

Bonus catch from your screenshot: at the very top you’ve got a broken helper block:

That’s wrong on two counts (it’s in a `PublicKey` extension and returns a `SecretKey`). Make it a `PublicKey` loader returning `PublicKey`. Same for the `SecretKey` one—each extension should load/save its own type.

If you want the lazy, keyboard-only way to clean this up from the project root:

Then build:

If it still complains, paste the **first** error line and I’ll point at the exact character that’s annoying the compiler.

Yes, sed is better for me. Less to go wrong

Perfect—let’s fix it with **sed** so your fingers don’t start a street fight with Xcode.

You’ve still got stray `SPHINCS.` prefixes and a borked “PEM helpers” block. Run these **from the repo root** (`slhdsa-cli`):

1) Make all types top-level (no `SPHINCS.` prefixes)

2) Fix the prehash helper signature (the thing you highlighted)

3) Replace the broken “PEM helpers” block with a clean one

Your file currently has a `PublicKey` extension that returns a **SecretKey** (lol, no) and then a duplicate `SPHINCS.PublicKey` extension. Nuke that whole region and insert the correct pair:

```
bash# Delete everything from "// PEM helpers" down to "// Modes /
randomness"
sed -i 's|/^\\|\\| PEM helpers$/|^\\|\\| Modes \\| randomness$/d'
Sources/slhdsa/main.swift

# Reinsert a correct helpers block right after the PARAM_SET line
awk '1;/^let PARAM_SET = Parameter\\.shake_256f$/ && !p {print "\\n// PEM
helpers\\nextension SecretKey {\\n    static func loadPEM(_ pemPath: String)
throws -> SecretKey {\\n        let s = String(decoding: try
readFile(pemPath), as: UTF8.self)\\n        return try SecretKey(pem: s)\\n
}\\n    func savePEM(_ pemPath: String) throws { try writeText(pemPath, text:
self.pem) }\\n}\\nextension PublicKey {\\n    static func loadPEM(_ pemPath:
String) throws -> PublicKey {\\n        let s = String(decoding: try
readFile(pemPath), as: UTF8.self)\\n        return try PublicKey(pem: s)\\n
}\\n    func savePEM(_ pemPath: String) throws { try writeText(pemPath, text:
self.pem) }\\n}\\n\\n// Modes / randomness"; p=1}' Sources/slhdsa/main.swift >
/tmp/main.swift && mv /tmp/main.swift Sources/slhdsa/main.swift
```

Note: this assumes the **library provides** `init(pem:)` and `.pem` (it does). If your build later says “no member `pem`”, ping me and I’ll flip this to the alt API.

4) Fix **BLAKE3** call (static, not instance)

5) Build (for real this time)

If it still complains, copy the **first 10–15 lines** of the new error and I’ll point at the exact character that’s offending the compiler.

Fetching <https://github.com/apple/swift-asn1.git> from cache Fetched
<https://github.com/apple/swift-asn1.git> from cache (0.18s) Computing version for
<https://github.com/apple/swift-asn1.git> Computed <https://github.com/apple/swift-asn1.git> at
1.4.0 (0.20s) Creating working copy for <https://github.com/apple/swift-asn1.git> Working copy
of <https://github.com/apple/swift-asn1.git> resolved at 1.4.0 Creating working copy for
<https://github.com/apple/swift-crypto.git> Working copy of <https://github.com/apple/swift-crypto.git>
resolved at 3.15.0 Creating working copy for
<https://github.com/theapplegates/SwiftSPHINCS.git> Working copy of
<https://github.com/theapplegates/SwiftSPHINCS.git> resolved at main (13fce85) Creating
working copy for <https://github.com/thecoolwinter/SwiftBlake3.git> Working copy of
<https://github.com/thecoolwinter/SwiftBlake3.git> resolved at 0.1.0 Creating working copy for
<https://github.com/leif-ibsen/ASN1> Working copy of <https://github.com/leif-ibsen/ASN1>
resolved at 2.7.0 Creating working copy for <https://github.com/leif-ibsen/Digest> Working copy
of <https://github.com/leif-ibsen/Digest> resolved at 1.13.0 Creating working copy for
<https://github.com/leif-ibsen/BigInt> Working copy of <https://github.com/leif-ibsen/BigInt>
resolved at 1.21.0 error: exhausted attempts to resolve the dependencies graph, with the
following dependencies unresolved: * 'swiftsphincs' from <https://github.com/leif-ibsen/SwiftSPHINCS.git> thor3@Mac slhdsa-cli %

Yep, I see it in your log. You're resolving your fork just fine...

...but the solver still has **another** constraint that wants **leif-ibsen/SwiftSPHINCS**:

Translation: somewhere in your graph, something still references that URL. Could be your `Package.swift`, a stale pin, or a tool config. Here's the fastest way to bulldoze it.

1) Prove there's only one SwiftSPHINCS in your Package.swift

From repo root:

You should see **exactly one** URL (your fork) and **no** leif-ibsen lines. If you see both, delete the leif-ibsen line.

2) Check for hidden mirrors / pins

SwiftPM can remember mirrors and stale pins outside `.build`.

3) Purge caches & stale state (be thorough)

4) Resolve and build again

5) If it STILL mentions leif-ibsen...

You're almost certainly opening a different folder than the one you edited, or a transitive dep somewhere has hard-coded that URL. Two quick checks:

If some dependency is hardcoding the leif URL (unlikely, but possible), the **mirror** from step 2 wins anyway—keep it in place.

Bonus: quick sanity after it compiles

If the build drops more errors, paste the **first 10–15 lines** and I'll point to the exact line causing them.

```
grep -nEi 'SwiftSPHINCS|theapplegates/SwiftSPHINCS|leif-ibsen/SwiftSPHINCS'  
Package.swift 9:.package(url: "https://github.com/leif-ibsen/SwiftSPHINCS.git", from:  
"3.4.0"), 10:.package(url: "https://github.com/theapplegates/SwiftSPHINCS.git", branch:  
"main"), 17:.product(name: "SwiftSPHINCS", package: "SwiftSPHINCS"), thor3@Mac slhdsa-  
cli %
```

Yep — your grep output is the smoking gun:

You're declaring **two** SwiftSPHINCS deps. SwiftPM treats both as the same identity (`swiftsphincs`), so it melts down. Pick **one**.

Here's the no-brainer fix (keep your fork; delete the upstream line):

If you prefer upstream instead, do the opposite:

Sanity check you're clean:

Edge case: if **some other dependency** hard-requires the leif URL (rare), force it to your fork with a mirror:

After that, `swift build -c release` should finally produce `./.build/release/slhdsa`. If it still complains, paste the first 10–15 lines of the error. I'll point at the exact character that's being annoying.

warning: 'slhdsa-cli': found 1 file(s) which are unhandled; explicitly declare them as resources or exclude from the target /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift.bak /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:52:50: error: cannot find 'writeText' in scope 50 | return try PublicKey(pem: s) 51 | } 52 | func savePEM(_ pemPath: String) throws { try writeText(pemPath, text: self.pem) } | - error: cannot find 'writeText' in scope 53 | } 54 | /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:67:52: error: type 'PreHash' has no member 'shake256' 65 | } 66 | 67 | private func prehashKind(_ len: Int) -> PreHash { .shake256(length: len) } | - error: type 'PreHash' has no member 'shake256' 68 | 69 | func slh_sign(sk: SecretKey, message: Data, opts: SignOptions) throws -> Data { /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:73:22: error: value of type 'SecretKey' has no member 'signRandomized' 71 | case.pure: 72 | return (opts.rand == .rnd) 73 | ? try sk.signRandomized(message: message, context: opts.context) | - error: value of type 'SecretKey' has no member 'signRandomized' 74 | : try sk.signDeterministic(message: message, context: opts.context) 75 | case.prehash: /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:74:22: error: value of type 'SecretKey' has no member 'signDeterministic' 72 | return (opts.rand == .rnd) 73 | ? try sk.signRandomized(message: message, context: opts.context) 74 | : try sk.signDeterministic(message: message, context: opts.context) | - error: value of type 'SecretKey' has no member 'signDeterministic' 75 | case.prehash: 76 | let ph = prehashKind(opts.prehashLen) /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:78:22: error: value of type 'SecretKey' has no member 'signPreHashRandomized' 76 | let ph = prehashKind(opts.prehashLen) 77 | return (opts.rand == .rnd) 78 | ? try sk.signPreHashRandomized(message: message, preHash: ph, context: opts.context) | - error: value of type 'SecretKey' has no member 'signPreHashRandomized' 79 | : try sk.signPreHashDeterministic(message: message, preHash: ph, context: opts.context) 80 | } /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:79:22: error: value of type 'SecretKey' has no member 'signPreHashDeterministic' 77 | return (opts.rand == .rnd) 78 | ? try sk.signPreHashRandomized(message: message, preHash: ph, context: opts.context) 79 | : try sk.signPreHashDeterministic(message: message, preHash: ph, context: opts.context) | - error: value of type 'SecretKey' has no member 'signPreHashDeterministic' 80 | } 81 | } /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:86:23: error: value of type 'PublicKey' has no member 'verify' 84 | switch mode { 85 | case.pure: 86 | return try pk.verify(message: message, signature: signature, context: context) | - error: value of type 'PublicKey' has no member 'verify' 87 | case.prehash: 88 | let ph = prehashKind(prehashLen) /Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:89:23: error: value of type 'PublicKey' has no member

```
'verifyPreHash' 87 | case.prehash: 88 | let ph = prehashKind(prehashLen) 89 | return try
pk.verifyPreHash(message: message, signature: signature, preHash: ph, context: context) | -
error: value of type 'PublicKey' has no member 'verifyPreHash' 90 | } 91 | }
/Users/thor3/Documents/slhdsa-cli/Sources/slhdsa/main.swift:118:40: error: cannot find
'readFile' in scope 116 | @inline(__always) func loadCtx(_ arg: String?) throws -> Data? { 117 |
guard let arg = arg else { return nil } 118 | if arg.hasPrefix("@") { return try
readFile(String(arg.dropFirst())) } | - error: cannot find 'readFile' in scope 119 |
guard let d = parseHex(arg) else { throw CLLError.badArgs("Invalid hex in --ctx") } 120 | return
d
```

Alright Paul, you're 90% there. The red ink you pasted is just three buckets of pain:

1. you're missing the module import for the SPHINCS types,
2. a few tiny I/O helpers vanished (so `readFile` / `writeText` / `die` are "not in scope"), and
3. you're calling API case/method names that don't exist in the fork you pinned.

Let's fix all of it, fast.

A. Kill the warning about the backup file

SwiftPM sees your `.bak` inside `Sources/` and whines.

B. Make the compiler see the SPHINCS types (and SHAKE256)

At the very top of `Sources/slhdsa/main.swift`, **make sure these are present**:

(If you see `import CryptoKit` instead of `import Crypto`, that's fine on macOS too. Just have one of them.)

C. Put back the tiny file helpers you deleted

Drop this block **near the top** (right after the imports is perfect):

That kills all your `cannot find 'readFile'/'writeText'/'writeFile'/'die' in scope` errors in one go.

D. Fix PreHash case spelling for this fork

Your error says: `type 'PreHash' has no member 'shake256'`. Many versions spell it with an underscore.

Quick check:

If you see `case shake_256(length: Int)`, update your helper:

(If it actually shows `shake256(length:)`, keep your existing line. Use **exactly** what the grep shows.)

Also: you still have that old `prehashSHAKE256` function in your file. You can delete it, or leave it— **but** if you leave it, you must have `import Crypto` so `SHAKE256()` exists. It's safer to just remove the whole function since you're using `PreHash` now.

E. Use the method names that actually exist in this package

Your current code calls:

- `sk.signRandomized(...)`, `sk.signDeterministic(...)`
- `sk.signPreHashRandomized(...)`, `sk.signPreHashDeterministic(...)`
- `pk.verify(...)`, `pk.verifyPreHash(...)`

The fork you pinned doesn't expose those exact names. Let's discover the real ones:

You'll see one of these two shapes:

Variant A (unified API):

If you have this one, replace your wrappers with:

Variant B (split methods):

If that's what grep shows, keep your current wrappers (they match Variant B). Your earlier errors happened because the compiler didn't see the module at all, not necessarily because the methods didn't exist.